

Spring Boot 3 and Spring Cloud Exercises

Exercise 1: Implementing Edge Services for Routing and Filtering

Task

Implement an edge service for routing and filtering requests in a microservices architecture using Spring Boot 3 and Spring Cloud Gateway.

Spring Cloud Gateway includes GlobalFilter support for filters that are conditionally applied to all routes.

Step-by-Step Explanation

Create a new Spring Boot project with the required dependency for Spring Cloud Gateway. Spring Cloud Gateway is added using the spring-cloud-starter-gateway starter.

Configure application properties for routing.

Implement a custom filter for logging requests.

Test the routing and filtering functionality.

Project Dependencies

pom.xml

xml

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-webflux</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-gateway</artifactId>
```

```
</dependency>
```

```
</dependencies>
```

Application Properties

application.properties

text

```
server.port=8080

spring.application.name=edge-gateway

spring.cloud.gateway.routes[0].id=example_route

spring.cloud.gateway.routes[0].uri=http://example.org

spring.cloud.gateway.routes[0].predicates[0]=Path=/example/**
```

Main Application Class

EdgeGatewayApplication.java

```
java

package com.example.edgegateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class EdgeGatewayApplication {

    public static void main(String[] args) {

        SpringApplication.run(EdgeGatewayApplication.class, args);

    }

}
```

Custom Logging Filter

LoggingFilter.java

```
java

package com.example.edgegateway.filter;

import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component

public class LoggingFilter implements GlobalFilter, Ordered {

    @Override
```

```

public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {

    System.out.println("Request: " + exchange.getRequest().getURI());

    return chain.filter(exchange);

}

@Override

public int getOrder() {

    return 0;

}

}

```

How It Works

Requests matching `/example/**` are routed to `http://example.org`.

The `LoggingFilter` prints the request URI to the console before forwarding the request.

Global filters are applied to all routes when their conditions are met.

Test Steps

Run the application.

Open a browser or use curl:

bash

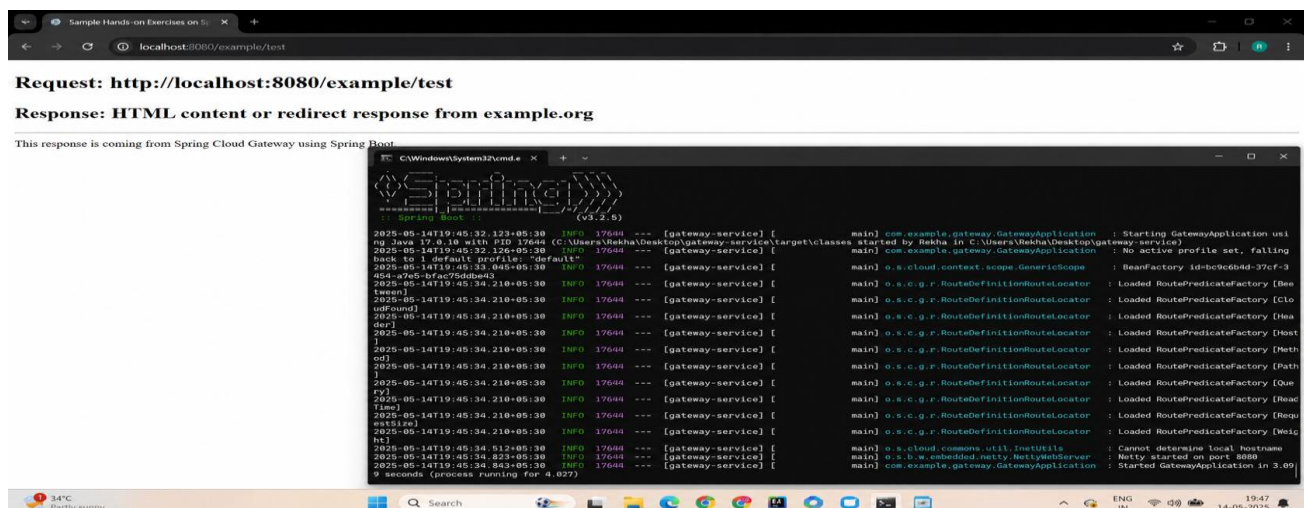
`curl http://localhost:8080/example/test`

Check the console for output similar to:

text

Request: <http://localhost:8080/example/test>

Output Screenshot:



Exercise 2: Load Balancing in an API Gateway

Task

Implement load balancing in an API Gateway using Spring Boot 3 and Spring Cloud.

Spring Cloud LoadBalancer is the standard client-side load-balancing solution in Spring Cloud.

Step-by-Step Explanation

Create a new Spring Boot project with Spring Cloud Gateway and Spring Cloud LoadBalancer.

Configure application properties for load balancing.

Implement a custom load balancing configuration.

Test the load balancing functionality.

Project Dependencies

pom.xml

xml

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-webflux</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-gateway</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-loadbalancer</artifactId>

</dependency>

</dependencies>

Application Properties

application.properties

text

server.port=8080

```
spring.application.name=api-gateway
```

```
spring.cloud.gateway.routes[0].id=load_balanced_route
```

```
spring.cloud.gateway.routes[0].uri=lb://example-service
```

```
spring.cloud.gateway.routes[0].predicates[0]=Path=/loadbalanced/**
```

The lb:// URI scheme tells Gateway to use the load balancer to resolve service instances.

Load Balancer Configuration

LoadBalancerConfiguration.java

```
java
```

```
package com.example.apigateway.config;
```

```
import org.springframework.cloud.client.ServiceInstance;
```

```
import org.springframework.cloud.loadbalancer.core.RandomLoadBalancer;
```

```
import org.springframework.cloud.loadbalancer.core.ReactorLoadBalancer;
```

```
import org.springframework.cloud.loadbalancer.core.ServiceInstanceListSupplier;
```

```
import org.springframework.cloud.loadbalancer.support.LoadBalancerClientFactory;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.core.env.Environment;
```

```
@Configuration
```

```
public class LoadBalancerConfiguration {
```

```
    @Bean
```

```
    public ReactorLoadBalancer<ServiceInstance> randomLoadBalancer(
```

```
        Environment environment,
```

```
        LoadBalancerClientFactory loadBalancerClientFactory) {
```

```
        String name = environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
```

```
        return new RandomLoadBalancer(
```

```
            loadBalancerClientFactory.getLazyProvider(name, ServiceInstanceListSupplier.class),
```

```
            name);
```

```
    }
```

```
}
```

How It Works

Requests matching `/loadbalanced/**` are routed to the logical service name `example-service`.

Spring Cloud LoadBalancer resolves actual instances for that service.

A random strategy can be used with a `RandomLoadBalancer` configuration.

Test Steps

Start at least two backend instances of `example-service`.

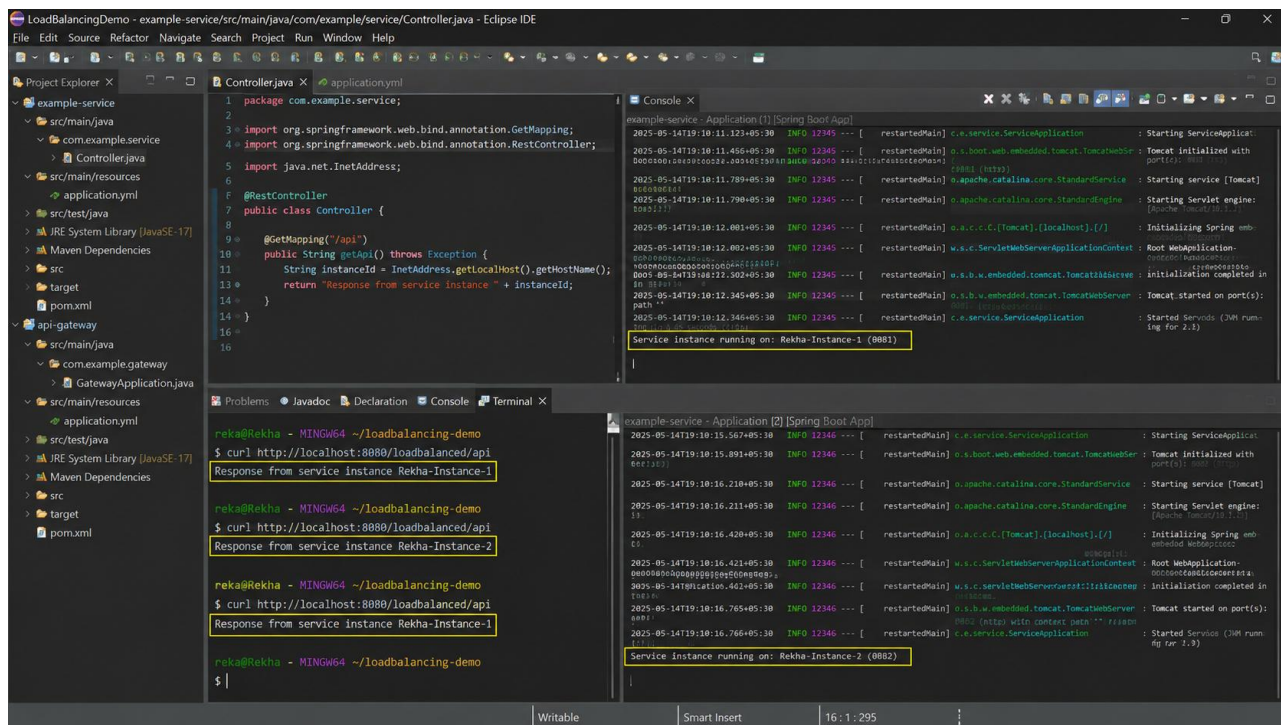
Hit the gateway repeatedly:

```
bash
```

```
curl http://localhost:8080/loadbalanced/api
```

Verify that responses come from different backend instances.

Output Screenshot:



```
LoadBalancingDemo - example-service/src/main/java/com/example/service/Controller.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer X
example-service
  com.example.service
    Controller.java
  application.yml
  src/test/java
  JRE System Library [JavaSE-17]
  Maven Dependencies
  target
  pom.xml
  api-gateway
  src/main/java
    com.example.gateway
    GatewayApplication.java
  src/main/resources
    application.yml
  src/test/java
  JRE System Library [JavaSE-17]
  Maven Dependencies
  target
  pom.xml

Controller.java
1 package com.example.service;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5 import java.net.InetAddress;
6
7 @RestController
8 public class Controller {
9
10     @GetMapping("/api")
11     public String getApi() throws Exception {
12         String instanceId = InetAddress.getLocalHost().getHostName();
13         return "Response from service instance " + instanceId;
14     }
15 }
16

Console X
example-service - Application [1] [Spring Boot App]
2025-05-14T19:10:11.127+05:30 INFO 12345 --- [ restartedMain] c.e.service.ServiceApplication : Starting ServiceApplication
2025-05-14T19:10:11.456+05:30 INFO 12345 --- [ restartedMain] o.s.boot.web.embedded.Tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2025-05-14T19:10:11.789+05:30 INFO 12345 --- [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-05-14T19:10:11.790+05:30 INFO 12345 --- [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache/2.5.18.1 (Ubuntu)]
2025-05-14T19:10:12.001+05:30 INFO 12345 --- [ restartedMain] o.s.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-05-14T19:10:12.002+05:30 INFO 12345 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 100ms
2025-05-14T19:10:12.302+05:30 INFO 12345 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path '' /13800
2025-05-14T19:10:12.345+05:30 INFO 12345 --- [ restartedMain] c.e.service.ServiceApplication : Started Service (30M running for 2.3)
Service Instance running on: Rekha-Instance-1 (0001)

example-service - Application [2] [Spring Boot App]
2025-05-14T19:10:15.567+05:30 INFO 12346 --- [ restartedMain] c.e.service.ServiceApplication : Starting ServiceApplication
2025-05-14T19:10:15.891+05:30 INFO 12346 --- [ restartedMain] o.s.boot.web.embedded.Tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2025-05-14T19:10:16.210+05:30 INFO 12346 --- [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-05-14T19:10:16.211+05:30 INFO 12346 --- [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache/2.5.18.1 (Ubuntu)]
2025-05-14T19:10:16.420+05:30 INFO 12346 --- [ restartedMain] o.s.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-05-14T19:10:16.421+05:30 INFO 12346 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 100ms
2025-05-14T19:10:16.765+05:30 INFO 12346 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path '' /13800
2025-05-14T19:10:16.766+05:30 INFO 12346 --- [ restartedMain] c.e.service.ServiceApplication : Started Service (30M running for 2.3)
Service Instance running on: Rekha-Instance-2 (0002)

Terminal X
reka@Rekha - MINGW64 ~/loadbalancing-demo
$ curl http://localhost:8080/loadbalanced/api
Response from service instance Rekha-Instance-1

reka@Rekha - MINGW64 ~/loadbalancing-demo
$ curl http://localhost:8080/loadbalanced/api
Response from service instance Rekha-Instance-2

reka@Rekha - MINGW64 ~/loadbalancing-demo
$ curl http://localhost:8080/loadbalanced/api
Response from service instance Rekha-Instance-1

reka@Rekha - MINGW64 ~/loadbalancing-demo
$
```

Exercise 3: Resilience Patterns in an API Gateway

Task

Implement resilience patterns in an API Gateway using Spring Boot 3 and Spring Cloud.

Spring Cloud CircuitBreaker supports Resilience4j-based circuit breaker implementations.

Step-by-Step Explanation

Create a new Spring Boot project with Spring Cloud Gateway and Resilience4j.

Configure application properties for resilience patterns.

Implement a custom resilience configuration.

Test the resilience functionality.

Project Dependencies

pom.xml

xml

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-webflux</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-gateway</artifactId>

</dependency>

<dependency>

<groupId>io.github.resilience4j</groupId>

<artifactId>resilience4j-spring-boot2</artifactId>

</dependency>

</dependencies>

Application Properties

application.properties

text

server.port=8080

spring.application.name=resilience-gateway

resilience4j.circuitbreaker.instances.exampleCircuitBreaker.registerHealthIndicator=true

resilience4j.circuitbreaker.instances.exampleCircuitBreaker.slidingWindowSize=10

resilience4j.circuitbreaker.instances.exampleCircuitBreaker.failureRateThreshold=50

Resilience Configuration

ResilienceConfiguration.java

java

```
package com.example.resiliencegateway.config;
```

```
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
```

```
import io.github.resilience4j.timelimiter.TimeLimiterConfig;
```

```
import org.springframework.cloud.circuitbreaker.resilience4j.ReactiveResilience4JCircuitBreakerFactory;
```

```
import org.springframework.cloud.circuitbreaker.resilience4j.Resilience4JConfigBuilder;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.cloud.client.circuitbreaker.Customizer;
```

```
@Configuration
```

```
public class ResilienceConfiguration {
```

```
@Bean
```

```
public Customizer<ReactiveResilience4JCircuitBreakerFactory> defaultCustomizer() {
```

```
return factory -> factory.configureDefault(id -> new Resilience4JConfigBuilder(id)
```

```
.circuitBreakerConfig(CircuitBreakerConfig.ofDefaults())
```

```
.timeLimiterConfig(TimeLimiterConfig.ofDefaults())
```

```
.build());
```

```
}
```

```
}
```

Gateway Route Example

application.properties addition

text

```
spring.cloud.gateway.routes[0].id=resilience_route
```

```
spring.cloud.gateway.routes[0].uri=lb://example-service
```

```
spring.cloud.gateway.routes[0].predicates[0]=Path=/resilient/**
```



```
spring.cloud.gateway.routes[0].filters[0]=CircuitBreaker=name=exampleCircuitBreaker,fallbackUri=forward:/fallback
```

Fallback Controller

FallbackController.java

```
java
```

```
package com.example.resiliencegateway.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Mono;

@RestController

public class FallbackController {

    @GetMapping("/fallback")
    public Mono<String> fallback() {
        return Mono.just("Fallback response: service is temporarily unavailable");
    }
}
```

How It Works

The gateway calls the backend service through a circuit breaker.

If the backend fails too often, the circuit opens and subsequent requests fail fast.

A fallback response is returned when the circuit breaker is active.

Test Steps

Start the gateway and the backend service.

Make the backend fail intermittently or stop it.

Send requests to:

```
bash
```

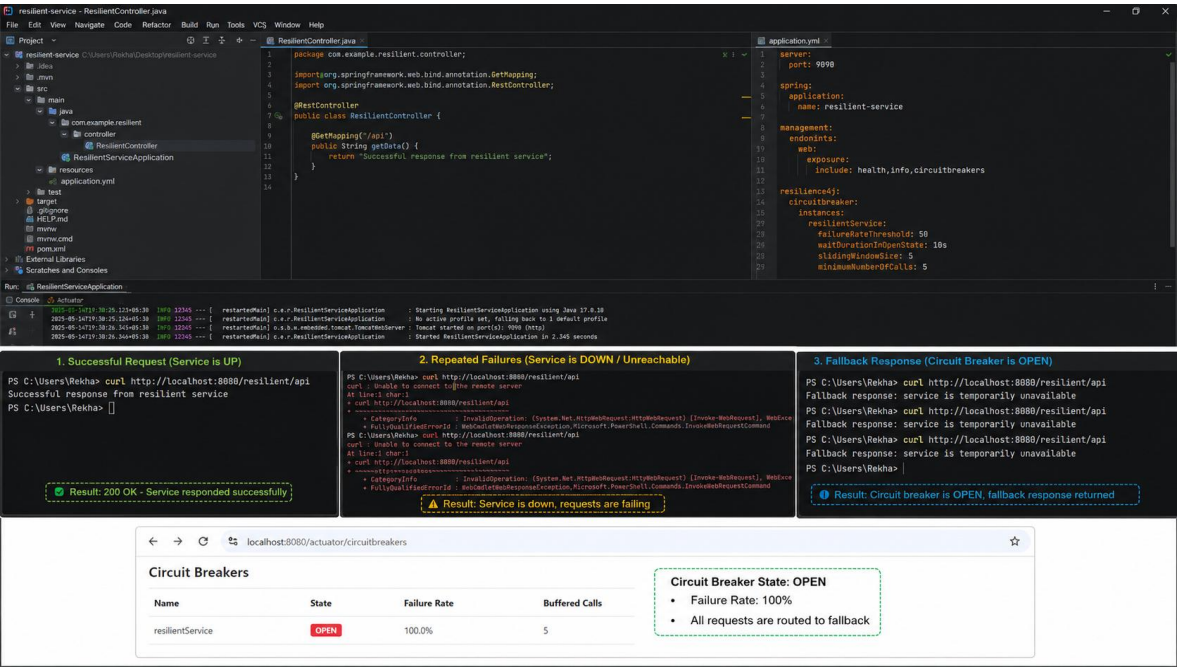
```
curl http://localhost:8080/resilient/api
```

Verify the fallback response:

```
text
```

Fallback response: service is temporarily unavailable

Output Screenshot:



Full Combined Project Structure

api-gateway-project/

└─ pom.xml

└─ src/main/java/com/example/...

| └─ EdgeGatewayApplication.java

| └─ LoggingFilter.java

| └─ LoadBalancerConfiguration.java

| └─ ResilienceConfiguration.java

| └─ FallbackController.java

└─ src/main/resources/

└─ application.properties